

Regression Analysis and Model Comparison for California Housing Price Prediction

1 Dataset Selection

The California Housing Dataset was selected for this regression task. It is available through the Scikit-learn library and represents a real-world problem of predicting housing prices.

Dataset Overview

Attribute	Description
Source	Scikit-learn dataset repository
Features	Income, house age, rooms, bedrooms, population, occupancy, latitude, longitude
Target	Median house value
Total Rows	20,640

1.1 Sample Data Instances

Index	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Lat	Long	Target
0	8.3252	41	6.98	1.02	322	2.56	37.88	-122.23	4.526
1	8.3014	21	6.24	0.97	2401	2.11	37.86	-122.22	3.585
2	7.2574	52	8.29	1.07	496	2.80	37.85	-122.24	3.521
3	5.6431	52	5.82	1.07	558	2.55	37.85	-122.25	3.413
4	3.8462	52	6.28	1.08	565	2.18	37.85	-122.25	3.422

Explanation: The table presents sample records from the dataset, showing how input features and the target variable are structured. It helps in understanding data variation and feature ranges.

2 Dataset Visualization

2.1 Feature Distribution

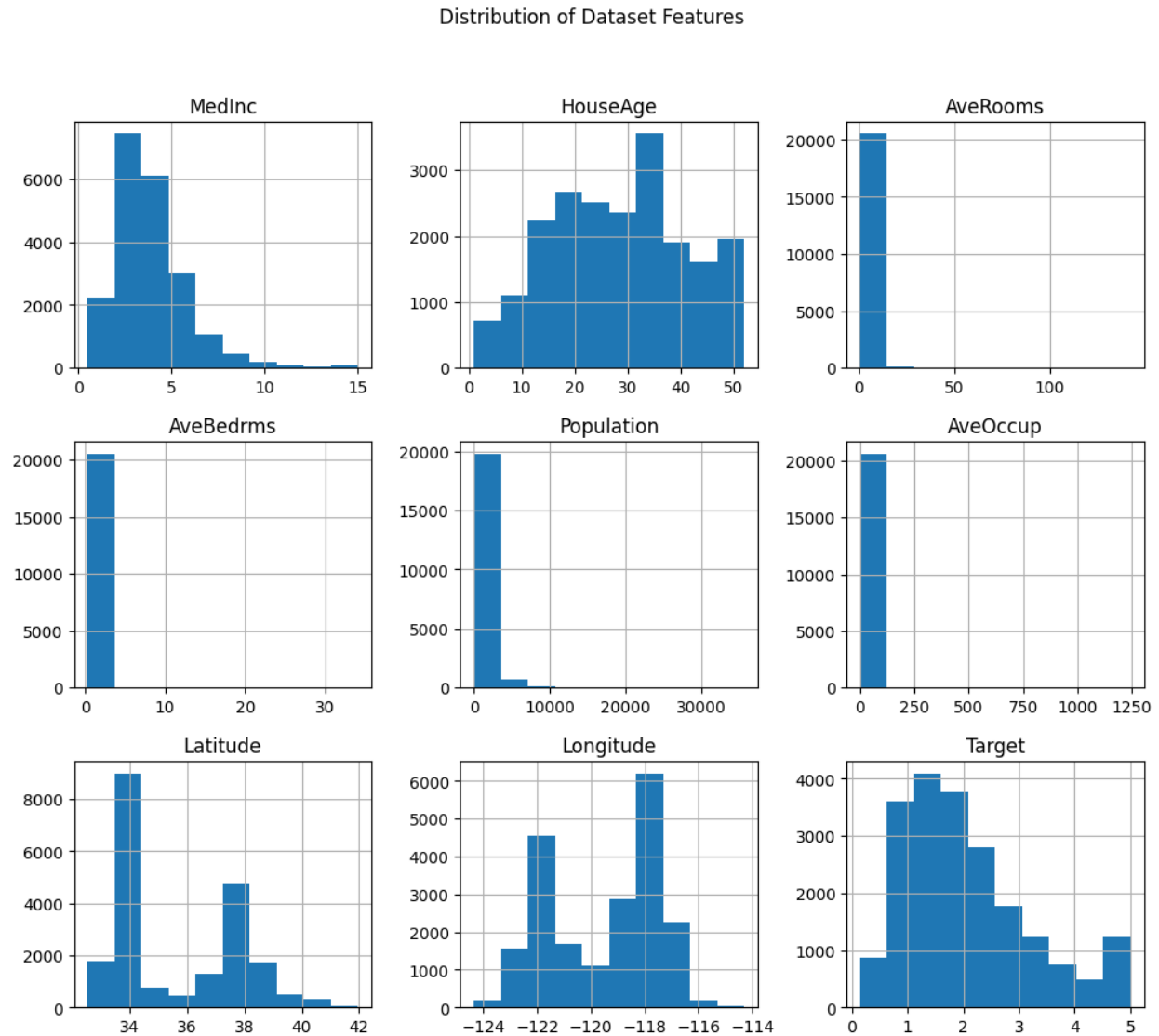


Figure 1: Distribution of Dataset Features

Explanation: The histogram shows the distribution of all features, highlighting differences in scale and skewness. This justifies preprocessing steps such as feature scaling.

3 Data Preprocessing

3.1 Missing Values

No missing values were found.

3.2 Feature Scaling

StandardScaler was applied.

Reason: Scaling ensures all features contribute equally and improves Linear Regression performance.

3.3 Train-Test Split

80% training and 20% testing split ensures fair evaluation.

4 Model Implementation

4.1 Code

```
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor

data = fetch_california_housing()
X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

lr = LinearRegression()
rf = RandomForestRegressor(n_estimators=100, random_state=42)

lr.fit(X_train_scaled, y_train)
rf.fit(X_train, y_train)

y_pred_lr = lr.predict(X_test_scaled)
y_pred_rf = rf.predict(X_test)
```

Explanation: The dataset is loaded and split into training and testing sets. Feature scaling is applied for Linear Regression. Two models are trained: Linear Regression (linear model) and Random Forest (ensemble model). Predictions are generated using both models.

5 Model Evaluation

```
from sklearn.metrics import mean_absolute_error, mean_squared_error,
    r2_score
import numpy as np

def evaluate(y_test, y_pred):
    mae = mean_absolute_error(y_test, y_pred)
    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, y_pred)
    return mae, mse, rmse, r2
```

Explanation: This function computes evaluation metrics including MAE, MSE, RMSE, and R^2 , which measure prediction error and model performance.

5.1 Results

Model	MAE	MSE	RMSE	R^2
Linear Regression	0.53	0.55	0.74	0.58
Random Forest	0.33	0.25	0.50	0.81

Analysis: Random Forest shows lower errors and higher R^2 , indicating better predictive performance.

6 Visualization and Analysis

6.1 Actual vs Predicted Values

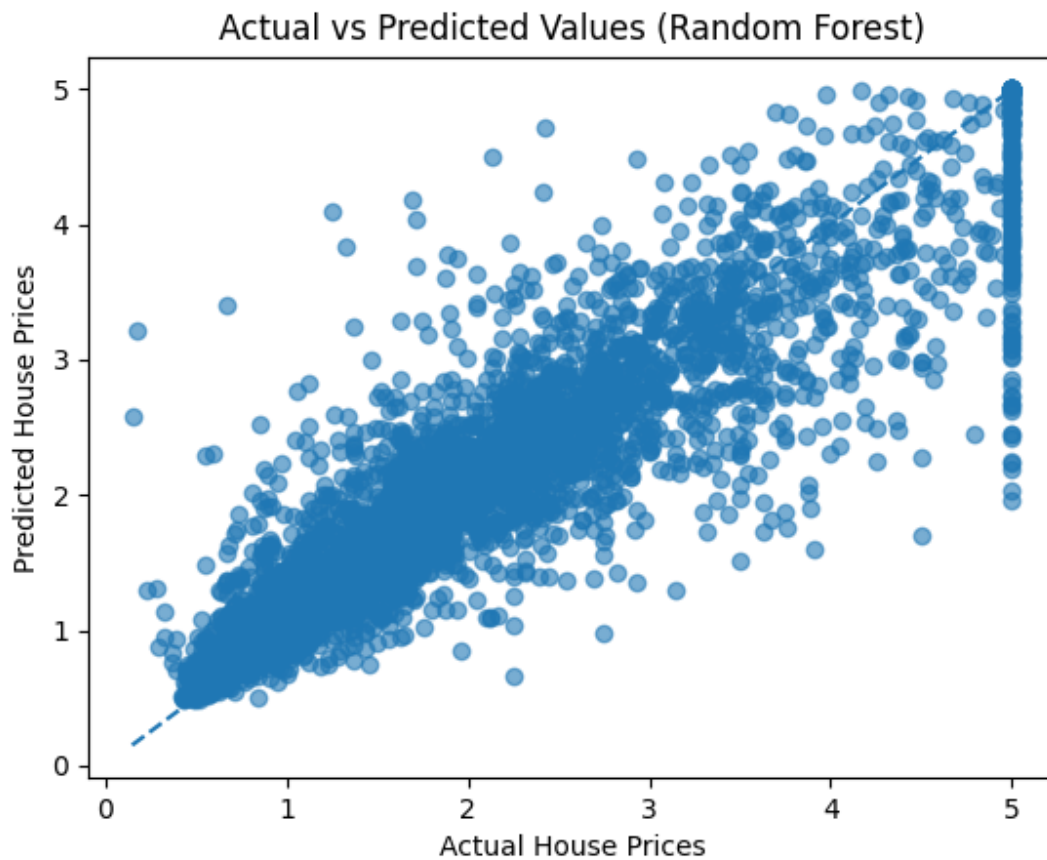


Figure 2: Actual vs Predicted House Prices

Explanation: Points close to the diagonal indicate accurate predictions. The model captures the overall trend effectively.

6.2 Actual vs Predicted (Line Comparison)

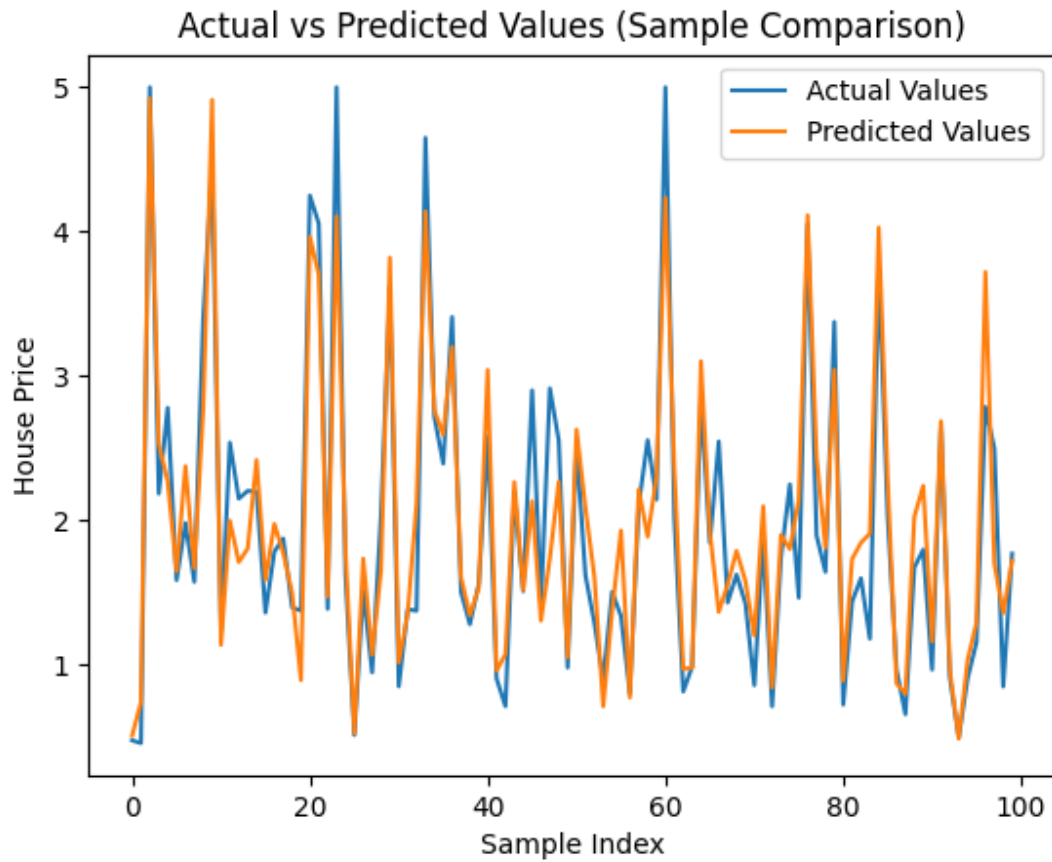


Figure 3: Actual vs Predicted Comparison

Explanation: Predicted values closely follow actual values, showing consistent performance.

6.3 Feature Importance

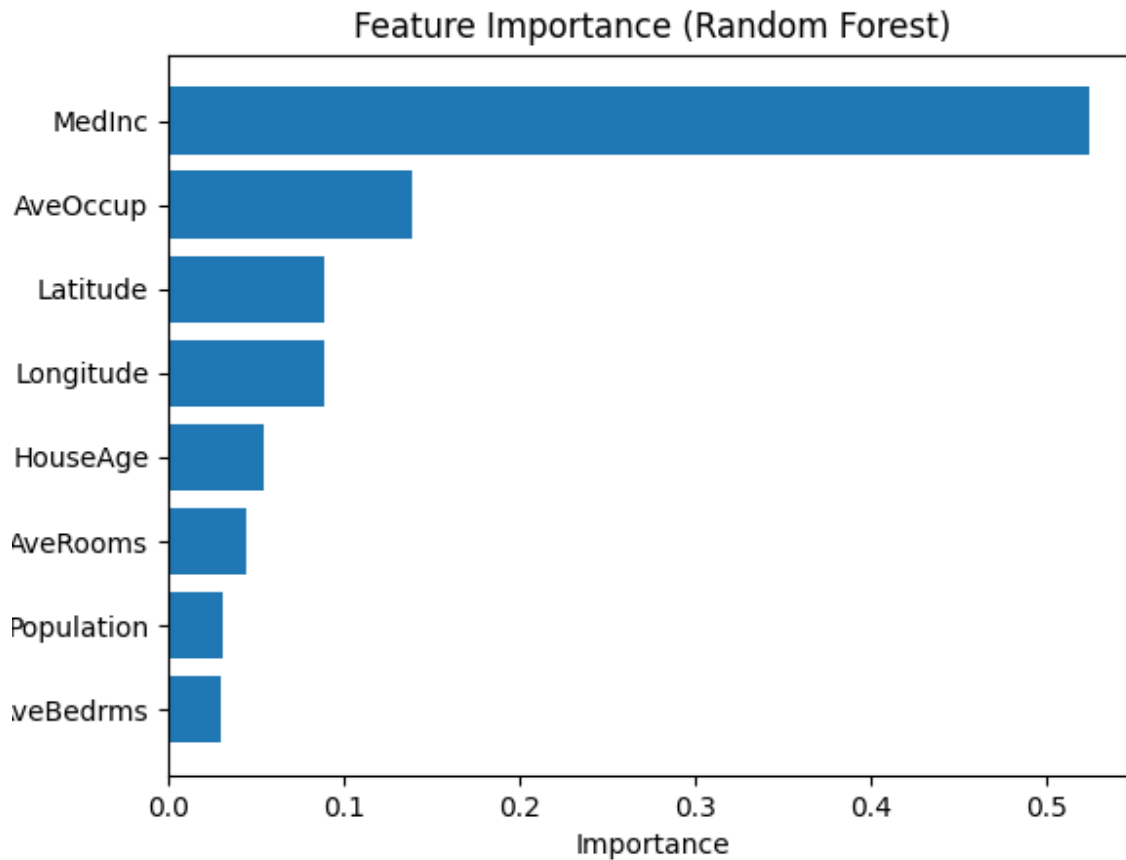


Figure 4: Feature Importance

Explanation: Median income has the highest importance, indicating strong influence on house prices.

7 Final Model Justification

Random Forest Regression is selected as the best-performing model.

- Captures non-linear relationships
- Handles feature interactions
- Provides higher accuracy

8 Conclusion

The report demonstrates a complete regression analysis workflow. Random Forest outperforms Linear Regression due to its ability to model complex patterns and provide more accurate predictions.